`RecursiveCharacterTextSplitter` and `TokenTextSplitter`. Here we use, `RecursiveCharacterTextSplitter` because it splits text by characters while maintaining a small overlap between chunks. This overlap preserves context across chunk boundaries, which is essential for tasks like summarization or information retrieval. Once the document is split into chunks, each piece can be processed individually, allowing strategies like `MapReduce` to summarize or analyze the content effectively.

The code that performs this task is just a couple of lines long:

```
const splitter = new RecursiveCharacterTextSplitter({
    chunkSize: 1500,
    chunkOverlap: 200,
});

const documentChunks = await splitter.splitDocuments(documents);
```

Once we have all the splits of the text, we can perform the summarization strategies we introduced using the `MapReduce` method.

APPLYING THE SUMMARIZATION CHAIN AND DISPLAYING TO THE USER

The core process of facilitating the summarization of large documents by using various techniques, including the MapReduce method, is encapsulated within the `loadSummarizationChain` function in LangChain.js. This function takes an instance of an LLM and an options object that specifies the desired summarization type, such as `"map_reduce"`, `"refine"`, or `"stuff"`. The following listing shows the code that we use to perform the summarization using MapReduce.

Listing 7.2 Performing summarization using MapReduce

```
import { loadSummarizationChain } from "langchain/chains";

const chain = loadSummarizationChain(model, {
  type: "map_reduce",
  verbose: true,
});

// Call the summarization chain
const res = await chain.invoke({
  input_documents: docs,
});

console.log("Summarization Result: ", res);
/*
Summarization Result:  {
  text: `Hadley Wickham's "Tidy Data" advocates for a
standardized data structure—each variable in a column,
each observation in a row, each unit in a table—to simplify
 data analysis.  This "tidy data" framework, illustrated
through numerous examples and contrasted with messy data
formats, improves data cleaning, manipulation, and
visualization, especially within R's vectorized environment.
  While acknowledging limitations and ongoing challenges
```

Initializes the summarization chain by calling loadSummarizationChain with the specified map_reduce type

Invokes the summarization chain asynchronously with the input documents (docs) to generate a summary